

Introdução ao Linux

Diretórios, comandos, pacotes e processos na prática

Weslei Ferreira Santos

MATA58 · Sistemas Operacionais
Instituto de Computação / UFBA

Agenda

1. Terminal: o palco onde tudo acontece
2. Distribuições Linux
3. Gerenciamento de pacotes
4. Sistema de arquivos, diretórios e permissões
5. Comandos básicos do terminal
6. Gerenciamento de processos
7. Criação de processos: fork, exec e wait
8. **Laboratório guiado**: descobrindo processos na prática
9. Scripts: um gostinho de automação

Terminal

O terminal

- Também chamado de **linha de comando** ou **shell**.
- Antigamente, era a *única* forma de interagir com o computador.
- Ainda hoje é essencial para gerenciar sistemas baseados em UNIX:
 - Servidores
 - Dispositivos embarcados (smartphones, computador de bordo, IoT)

Ideia-chave para hoje

O shell também é um processo. Quando você digita um comando, está pedindo ao sistema operacional para criar e gerenciar outro processo.

Distribuições Linux

O que são distribuições Linux?

- Uma **distribuição** (“distro”) combina o kernel Linux com um conjunto de softwares, aplicativos e ferramentas.
- Desenvolvidas e mantidas por comunidades, organizações ou empresas.
- Cada uma tem seu próprio: gerenciamento de pacotes, ambiente gráfico, configurações e foco de uso.

Lançado em 2004 pela **Canonical Ltd.** Foco em ser fácil de usar, estável e seguro, para iniciantes e avançados.

- **Ambiente gráfico:** GNOME por padrão.
- **Pacotes:** APT + Software Center.
- **Versões:** LTS (2 em 2 anos, suporte de 5 anos) e intermediárias (6 em 6 meses, suporte de 9 meses).

Patrocinado pela **Red Hat**, desenvolvido pela comunidade Fedora Project. Base para o RHEL.

- **Lançamentos:** ciclo de ~ 6 meses, sempre com tecnologia recente.
- **Pacotes:** DNF (evolução do YUM).
- **Segurança:** SELinux integrado.
- **Edições:** Workstation, Server e spins (KDE, Xfce, LXQt...).

Lançado em 1993 por Ian Murdock. Base de Ubuntu, Linux Mint e elementary OS.

- **Estabilidade:** rigoroso controle de qualidade antes de entrar na versão *stable*.
- **Repositórios:** main (livre), contrib, non-free.
- **Pacotes:** APT + formato `.deb`.
- Suporta várias arquiteturas: amd64, i386, ARM, MIPS, s390x, PowerPC...

Lançado em 2002 por Judd Vinet. Filosofia **KISS** (Keep It Simple, Stupid).

- **Rolling release:** atualizações contínuas, sem versões fixas.
- **Pacman:** gerenciador de pacotes rápido e eficiente.
- **AUR:** repositório comunitário de pacotes extra.
- **ArchWiki:** uma das melhores documentações do ecossistema Linux.

Conjuntos de software que fornecem interface gráfica e experiência de usuário consistente.

GNOME	Moderno e minimalista; foco em simplicidade e produtividade. Padrão em Fedora, Debian e Ubuntu.
KDE Plasma	Altamente configurável, rico em recursos, experiência similar ao Windows.
Xfce	Leve e rápido; ótimo para hardware mais antigo ou limitado.
LXQt	Evolução do LXDE com Qt; extremamente eficiente em recursos.

Servidores gráficos: X11 vs. Wayland

Intermediários entre aplicativos e hardware gráfico: gerenciam exibição, janelas e dispositivos de entrada.

X.Org Server (X11)

- O mais usado e estabelecido.
- Arquitetura modular, suporta conexão remota.
- Antigo, com limitações de desempenho.

Wayland

- Mais moderno, busca substituir o X11.
- Melhor desempenho, segurança e isolamento.
- Adotado por GNOME e KDE, mas ainda com lacunas de compatibilidade.

Gerenciamento de pacotes

O que é um pacote?

- Conjunto de arquivos e metadados necessários para **instalar, atualizar ou remover** um software.
- Inclui: código executável, configurações, bibliotecas, documentação e **dependências**.
- Formatos comuns: `.deb` (Debian/Ubuntu), `.rpm` (Red Hat/Fedora), `.tar.gz` (tarball).

Gerenciador de pacotes

Ferramenta que automatiza download, instalação, atualização e remoção de pacotes, resolvendo dependências e conflitos.

Pacotes dependentes de distribuição

	.deb		.rpm		Pacman
Distros	Debian, Ubuntu, Mint		Fedora, RHEL, CentOS		Arch, Manjaro
Gerente	dpkg + apt		rpm + dnf/yum		pacman
Extra	repositórios main/contrib/non-free		scripts pré/pós- instalação		AUR (comunitário)

Pacotes independentes de distribuição

Empacotam o software **com todas as dependências**, rodando em qualquer distro compatível.

Flatpak (projeto GNOME)

- Isolamento/sandboxing
- Atualizações atômicas
- Sistema de portais para acesso a recursos

Snap (Canonical)

- Ambiente confinado
- Snap Store centralizada
- Múltiplos canais (stable, beta, edge...)

Pesquisando e instalando pacotes

```
# APT (Debian/Ubuntu)
```

```
$ apt search <pacote>    $ apt show <pacote>    $ apt list --installed
```

```
# DNF (Fedora)
```

```
$ dnf search <pacote>    $ dnf info <pacote>    $ dnf list installed
```

```
# Pacman (Arch)
```

```
$ pacman -Ss <pacote>    $ pacman -Si <pacote>    $ pacman -Q
```

```
# Flatpak / Snap
```

```
$ flatpak search <pacote>    $ snap find <pacote>
```

Sistema de arquivos e diretórios

Hierarquia do sistema de arquivos (FHS)

- O **FHS** (Filesystem Hierarchy Standard) define onde encontrar e armazenar arquivos, independentemente da distribuição.
- No boot, o sistema de arquivos raiz é disponibilizado: / (diretório raiz).
- Todos os demais diretórios e arquivos são acessíveis a partir dele.

Diretórios importantes (1/2)

/bin e /sbin

Programas (binários) do sistema.

/dev

Arquivos de dispositivos.

/etc

Arquivos de configuração do sistema.

/lib e /lib64

Bibliotecas para os binários de /bin e /sbin.

/boot

Imagem do kernel e gerenciador de boot.

/home

Arquivos de usuário: maior partição, pode ficar em disco separado.

Diretórios importantes (2/2)

/tmp

Arquivos temporários.

/var

Logs, filas de impressão/e-mail e outros arquivos dinâmicos.

/proc

Diretório virtual: acesso a variáveis e recursos do sistema.

/usr

Comandos de usuário, código-fonte, documentação.

/opt

Pacotes específicos fora do padrão (ex.: KDE, OpenOffice).

/usr/local

Software local, compilado manualmente.

Permissões: conceito

As permissões definem **quem pode acessar** determinados arquivos ou diretórios, mantendo segurança e organização.

- Cada arquivo/diretório tem um **proprietário** (owner) e um **grupo** associado.
- Um **grupo** é um conjunto de usuários que compartilham as mesmas permissões sobre um arquivo, sem precisar liberar acesso individualmente para cada um.
- Por padrão, apenas o dono pode alterar o arquivo.
- Existe a distinção **Usuário Comum** × **Super Usuário** (root).
- `chown` muda o proprietário; `chgrp` muda o grupo; `chmod` altera as permissões.

Três categorias, três permissões

As permissões são divididas em três categorias, cada uma com três tipos:

Categoria	Tipo de permissão
Proprietário (owner)	r (Read): ler / listar
Grupo	w (Write): modificar / criar / apagar
Outros	x (Execute): executar / entrar no diretório

Outros = todo usuário que não é o dono nem membro do grupo.

Representando permissões

As permissões aparecem como uma sequência de caracteres ou como 3 dígitos octais:

```
-rw-r--r--    equivalente a    644
```

- 1º caractere: tipo (- arquivo, d diretório, l link).
- Depois, 3 grupos de 3: proprietário, grupo, outros.
- Cada letra presente ou - (ausente): r, w, x.

Entendendo o chmod numérico

```
$ chmod 755 arquivo
```

Cada dígito é a soma de: **r=4**, **w=2**, **x=1**.

	Dígito	Binário	Permissão
Proprietário	7	111	rwX (4+2+1)
Grupo	5	101	r-X (4+0+1)
Outros	5	101	r-X (4+0+1)

`chmod 755` → dono lê/escreve/executa; grupo e outros só leem/executam.

Exemplo prático de leitura

```
$ ls -l  
drwxr-xr-x 1 wesleiferreira wesleiferreira 6 mar 8 20:59 Android
```

- `d` → é um diretório.
- `rwxr-xr-x` → dono: `rw`; grupo: `r-x`; outros: `r-x`.
- `wesleiferreira / wesleiferreira` → proprietário e grupo.
- `6` → tamanho em KB; datas de modificação; nome do diretório.

Comandos básicos do terminal

Duas formas básicas de descobrir como usar um comando:

```
$ comando --help      # resumo rapido de uso
$ man comando         # manual completo
```

Navegação: pwd, cd, ls

```
$ pwd                # mostra o diretório atual
$ cd Downloads       # entra em Downloads
$ cd                 # sem argumento: vai para o home
$ cd ..              # sobe um nível
$ ls                  # lista arquivos/pastas
$ ls -la             # com ocultos (-a) e detalhes (-l)
```

Criando e removendo diretórios

```
$ mkdir aula                # cria um diretorio
$ mkdir -p aula/sobre/linux # cria a cadeia inteira
$ rmdir aula                # remove SE estiver vazio
$ rm -r aula                # remove com todo o conteudo
```

Cuidado

`rm -r` apaga permanentemente. Não há lixeira no terminal.

Copiando, movendo, renomeando

```
$ cp arquivo.zip Desktop/      # copia
$ cp -r pasta/ destino/        # copia recursivamente
$ mv nome.zip nomeNovo.zip     # renomeia
$ mv arquivo.txt ../pratica/    # move para outro diretorio
$ touch novo.txt                # cria arquivo vazio
```

Praticando em sequência

```
$ mkdir aula && cd aula      # cria a pasta e ja entra nela
$ nano arquivo.txt          # abre o editor e cria o arquivo
$ cat arquivo.txt           # exhibe o conteudo do arquivo
$ grep segunda arquivo.txt   # busca a palavra "segunda" no arquivo
$ mkdir ../pratica          # cria uma pasta irma, um nivel acima
$ cp arquivo.txt ../pratica/ # copia o arquivo para a outra pasta
$ mv arquivo.txt outro_nome.txt # renomeia o arquivo
$ rm outro_nome.txt         # apaga o arquivo renomeado
```

Executando dois comandos seguidos

Sem dependência (executa os dois, não importa o resultado):

```
$ comando1 ; comando2
```

Com dependência (só executa o segundo se o primeiro der certo):

```
$ comando1 && comando2  
$ mkdir pasta && cd pasta
```

Utilitários de texto: cat e grep

```
$ cat file1.txt file2.txt > combined.txt    # exibe/combina  
$ grep "error" log.txt                      # busca padrao
```

cat exibe e combina arquivos de texto. grep procura padrões (inclusive expressões regulares).

Utilitários de texto: sed

sed (stream editor) edita um fluxo de texto linha por linha, sem abrir um editor.

```
$ sed 's/old/new/g' input.txt > output.txt
# troca "old" por "new" em cada linha (g = todas as ocorrencias)

$ sed -n '2,4p' arquivo.txt
# imprime SOMENTE as linhas de 2 a 4 (-n desliga a saida padrao)

$ sed '/erro/d' log.txt
# apaga (d) as linhas que contem a palavra "erro"
```

Utilitários de texto: awk

awk processa dados organizados em colunas: \$1 é a 1ª coluna, \$2 a 2ª, e assim por diante.

```
$ awk '{print $1}' data.txt
# imprime somente a primeira coluna de cada linha

$ awk -F, '{print $2}' data.csv
# -F, define a virgula como separador; imprime a 2a coluna

$ awk '{sum += $1} END {print sum}' data.txt
# soma o valor da 1a coluna de cada linha e imprime o total
```

Utilitários de texto: cut

cut recorta pedaços de cada linha, por posição de caractere ou por campo delimitado.

```
$ cut -f1,3 -d, data.csv
# extrai os campos 1 e 3, separados por virgula (-d ,)

$ cut -c1-5 arquivo.txt
# extrai os 5 primeiros caracteres (-c) de cada linha

$ cut -f2 -d: /etc/passwd
# extrai o 2o campo de cada linha, separado por ":"
```

Utilitários de texto: sort, uniq, wc, tr

```
$ sort -n -r numbers.txt
# ordena numericamente (-n) e de forma decrescente (-r)

$ sort input.txt | uniq > output.txt
# remove linhas duplicadas ADJACENTES (por isso o sort antes)

$ wc -l file.txt
# conta linhas (-l); sem opcao, conta linhas/palavras/caracteres

$ tr '[:upper:]' '[:lower:]' < input.txt > output.txt
# traduz maiusculas em minusculas, caractere por caractere
```

Gerenciamento de processos

Programa vs. Processo

Programa

- Arquivo em disco
- Passivo, parado

Processo

- Programa em execução
- Usa CPU, memória, dispositivos

Definição

“Um processo é a forma de representar um programa em execução.” Ele utiliza recursos computacionais (processador, memória, armazenamento, rede, dispositivos de E/S) para realizar suas tarefas.

A pergunta central da aula

Se muitos programas querem CPU, memória e dispositivos ao mesmo tempo, **quem decide** a ordem, os limites e a proteção?

Os quatro estados essenciais

O Linux trabalha, essencialmente, com quatro situações de execução:

Estado	Significado
Executando (R)	Ativo, rodando ou pronto para rodar na CPU
Dormente (S)	Aguardando um evento ou temporização finita
Parado (T)	Suspenso, geralmente por um sinal do usuário
Zumbi (Z)	Terminou, mas o pai ainda não “tomou ciência”

Sobre o estado Zumbi

Um processo zumbi não ocupa memória, mas mantém uma entrada na tabela de processos até que o pai seja notificado e o libere.

Características de um processo

Todo processo carrega consigo:

- **Número identificador (PID)**
- **Proprietário** do processo
- **Estado** do processo
- **Prioridade** de execução
- **Recursos de memória** alocados

Identificadores: PID, PPID, UID, GID

PID (Process Identifier)

Número único que o sistema operacional atribui a cada processo para identificá-lo.

PPID (Parent Process Identifier)

Número que identifica o processo **pai** do processo (quem o criou).

UID / GID (User / Group Identifier)

Identificam, respectivamente, o usuário e o grupo proprietários do processo.

Listando processos: ps

Listar os processos filhos do terminal ativo:

```
$ ps
```

Para mostrar mais detalhes:

```
$ ps -f
```

Listar os processos de um usuário específico:

```
$ ps -u user  
$ ps -f -u user      # com mais detalhes
```

Listando todos os processos

Listar os processos de **todos** os usuários:

```
$ ps aux
```

Aplicando um filtro com *pipe*:

```
$ ps aux | grep nano
```

Listando todos os processos

Listar os processos de **todos** os usuários:

```
$ ps aux
```

Aplicando um filtro com *pipe*:

```
$ ps aux | grep nano
```

Modo interativo, atualizado em tempo real:

```
$ top
```

```
$ htop
```

O que são sinais?

Definição

Sinais são o meio usado para que processos possam se comunicar e para que o sistema possa interferir em seu funcionamento.

Controle	Finalização
STOP / CONT	TERM / ILL / KILL

Sinais mais usados no dia a dia

Sinal	Número	Efeito
SIGINT	2	Ctrl+C no primeiro plano
SIGTERM	15	kill PID: pedido educado de saída
SIGSTOP	19	Suspende; não pode ser ignorado
SIGCONT	18	Retoma um processo suspenso
SIGKILL	9	Último recurso, sem chance de limpeza

Atenção

`kill -STOP` não mata o processo: ele apenas muda o estado para **T** (parado). O processo continua na tabela.

Comandos kill e killall

O comando `kill` envia um sinal a um PID. Por padrão, executa o sinal `TERM`.

```
$ kill -STOP 4220  
$ kill -9 4220
```

Quando você não sabe o PID, mas sabe o nome do programa, use `killall`:

```
$ killall -STOP nano
```

Determinar a prioridade de um processo manualmente não é uma prática comum, pois o próprio Linux já faz isso bem. Mas é possível intervir:

```
$ nice -n -5 nano
```

O parâmetro varia de **-19** (maior prioridade) a **19** (menor prioridade).

Para alterar a prioridade de um processo já em execução:

```
$ renice +19 1000
```

Isso altera a prioridade do processo 1000.

Criação de processos: fork, exec e wait

De onde vêm os processos?

Todo processo (exceto o primeiro, criado pelo kernel no boot: `init/systemd`, PID 1) foi criado por outro processo. É por isso que todo processo tem um PPID.

```
$ pstree -p 1  
systemd(1)-+-sshd(842)---bash(1055)---pstree(2011)
```

A criação de um processo segue, classicamente, dois passos que podem ser usados juntos ou separados: `fork()` e `exec()`.

`fork()`: duplicando um processo

`fork()` é uma chamada de sistema que cria um novo processo (o filho) como uma cópia quase idêntica do processo que a chamou (o pai). Ambos continuam a execução exatamente no ponto seguinte à chamada de `fork()`, só que agora são dois processos independentes, com PIDs diferentes.

- O filho recebe uma cópia do espaço de memória, variáveis e arquivos abertos do pai (na prática, otimizada com *copy-on-write*).
- A partir do `fork()`, pai e filho são processos totalmente separados, cada um com seu próprio PID.
- O valor de retorno de `fork()` é o que permite ao código saber “quem ele é” depois da chamada.

O valor de retorno de `fork()` é a chave

Quem recebe	Valor retornado	Significado
Processo pai	PID do filho (positivo)	“Criei um filho com este PID.”
Processo filho	0	“Eu sou o filho.”
Erro	-1	<code>fork()</code> falhou.

fork() em código

```
pid = fork();
if (pid == 0) {
    // roda APENAS no processo filho
    printf("Sou o filho, meu PID e %d\n", getpid());
} else if (pid > 0) {
    // roda APENAS no processo pai
    printf("Sou o pai, criei o filho %d\n", pid);
} else {
    perror("fork");    // fork() falhou
}
```

Ponto de atenção clássico

Depois de um `fork()` bem-sucedido, existem DOIS processos executando o mesmo código a partir daquele ponto. A única forma de diferenciar é checando o valor retornado (`0` = filho, `>0` = pai).

`exec()`: trocando o programa em execução

Se `fork()` cria uma cópia do processo atual, a família `exec()` (`execve`, `execl`, `execvp`...) substitui completamente a imagem de memória do processo pelo código de um novo programa. O PID continua o mesmo, mas o programa em execução muda por completo.

- Depois de um `exec()` bem-sucedido, o código anterior nunca mais volta a executar naquele processo.
- É assim que o shell “executa” seus comandos: dá `fork()` para criar um filho, e o filho dá `exec()` para virar o programa pedido.
- Se o `exec()` falhar, ele retorna um erro e o código original continua dali.

O padrão fork + exec

Combinação clássica usada por todo shell Unix para rodar um novo programa sem deixar de existir:

```
pid = fork();

if (pid == 0) {
    // filho: vira o programa "ls -la"
    execlp("ls", "ls", "-la", NULL); // troca o programa em execucao
    perror("exec falhou"); // so chega aqui se falhar
    exit(1);
} else {
    // pai: continua sendo o shell
    printf("Aguardando o filho terminar...\n");
}
```

Por que separar `fork()` de `exec()`?

Porque isso dá ao processo pai (o shell) a chance de configurar o ambiente do filho **antes** de ele virar outro programa:

- Redirecionar entrada/saída padrão (operadores `>`, `<`, `|`);
- Mudar variáveis de ambiente;
- Ajustar prioridade com `nice`;

Tudo isso acontece no intervalo entre o `fork()` e o `exec()`.

`wait()` e `waitpid()`: sincronizando pai e filho

Depois do `fork()`, pai e filho rodam de forma independente e concorrente. Se o pai quiser saber quando (e como) o filho terminou, precisa chamar `wait()` ou `waitpid()`.

- `wait(&status)`: bloqueia o pai até QUALQUER filho terminar.
- `waitpid(pid, &status, opcoes)`: espera um filho específico, ou não bloqueia (`WNOHANG`).

```
pid_t filho = fork();
if (filho == 0) {
    execlp("sleep", "sleep", "2", NULL); // troca o programa em execucao
    exit(1);
} else {
    int status;
    wait(&status); // pai espera o filho terminar
}
```

Zumbis e órfãos revisitados

Com `fork()`, `exec()` e `wait()` explicados, os dois estados especiais de processo fazem muito mais sentido:

Processo zumbi

O filho terminou, mas o pai ainda não chamou `wait()`. O SO guarda o status de saída até ser “colhido”.

Processo órfão

O pai terminou antes do filho. O filho é adotado pelo `init/systemd`, que chamará `wait()` por ele.

Resumindo

Zumbi = filho terminou, pai ainda não colheu. Órfão = pai terminou antes, filho foi adotado. Um zumbi de longa duração é sinal de bug: um pai que nunca chama `wait()`.

Visualizando o ciclo completo

1. Pai chama `fork()`
-> cria copia identica: processo filho (novo PID, PPID = pai)
2. (Opcional) Filho chama `exec()`
-> filho vira um programa novo, mantendo o mesmo PID
3. Filho executa e, eventualmente, termina
-> vira ZUMBI ate que alguém colha seu status
- 4a. Pai chama `wait()/waitpid()` -> zumbi e removido
- 4b. Pai ja tinha terminado antes -> filho e adotado (orfao)

Da teoria para a prática

Você já sabe como um processo nasce: um pai chama `fork()` e cria um filho, que pode virar outro programa com `exec()`.



“Você já sabe como os processos nascem: repare nisso durante os experimentos a seguir.”

Nos próximos experimentos, você vai ver na prática pais, filhos, PPIDs e sinais: tudo o que acabamos de explicar em código.

Laboratório guiado

Atividade guiada de laboratório

Vamos observar processos reais: criação pelo shell, PID e PPID, estados, sinais e consumo de recursos.

Regra da aula

Primeiro observamos no terminal, depois nomeamos o conceito.

Ambiente

- Linux nativo, WSL, máquina virtual ou laboratório remoto.
- Terminal aberto e um editor simples para anotações.
- Permissão para executar `ps`, `kill` e `top` como usuário comum.

Regras de segurança

- Confirme sempre o PID antes de qualquer `kill`.
- Encerre apenas os processos criados por você.
- Não use `sudo` nesta atividade.

A trilha do laboratório: sete experimentos curtos

01 Primeiro plano: por que o terminal “trava”?

02 Segundo plano: o que o & muda?

03 Tabela de processos: o que o ps mostra?

04 Estado do processo: o que significa S ou T?

05 Hierarquia: quem criou quem?

06 Sinais: como pedir mudança ao SO?

07 Recursos: quem fica com a CPU?

Cada experimento leva de 2 a 4 minutos.

Experimento 1 de 7: Um programa ocupa o terminal

Um comando em primeiro plano prende a interação do shell até terminar.

```
$ sleep 300
# o terminal fica aguardando: nada de novo prompt.
# para interromper: Ctrl + C
$
```

Perguntas para a turma:

- O terminal travou ou está obedecendo a um programa?
- O que o Ctrl+C realmente fez com o processo?

Ideia-chave

O shell também é um processo: está apenas esperando o processo em primeiro plano terminar. Ctrl+C envia o sinal SIGINT.

Experimento 2 de 7: Rodando em segundo plano

Agora o processo existe, mas o shell continua disponível para novos comandos.

```
$ sleep 300 &
[1] 18423

$ jobs
[1]+  Running      sleep 300 &

$ fg %1          # traz de volta ao primeiro plano
# Ctrl+Z suspende e devolve o prompt
```

Não confunda os dois números

[1] é o número do **job**, controlado pelo shell. 18423 é o **PID**, controlado pelo sistema operacional.

Experimento 3 de 7: A tabela de processos

O comando `ps` mostra uma fotografia dos processos visíveis naquele instante.

```
$ ps -f
UID      PID    PPID  C  STIME TTY      TIME CMD
aluno 17110   17092  0  08:50 pts/1    00:00 bash
aluno 18423   17110  0  08:54 pts/1    00:00 sleep 300
aluno 18428   17110  0  08:54 pts/1    00:00 ps -f
```

Pergunta: qual processo é o pai do `sleep`? Compare o PPID do `sleep` com o PID do `bash`.

Experimento 4 de 7: O estado do processo

Filtramos por PID para reduzir ruído e ler o estado administrado pelo SO.

```
$ PID=18423
$ ps -o pid,ppid,state,cmd -p $PID
  PID  PPID S  CMD
18423 17110 S  sleep 300
```

Dica: guarde o PID em uma variável: assim, todos os comandos seguintes usam \$PID e evitam erro de digitação.

Experimento 5 de 7: Quem criou quem?

A árvore de processos torna visível a relação entre pai e filho.

```
$ pstree -p $$  
bash(17110)---sleep(18423)  
  
# alternativa sem pstree:  
$ ps -o pid,ppid,cmd -p $$,$PID
```

Experimento 6 de 7: Suspende, retomar, encerrar

Sinais são pedidos ao SO para que ele mude o estado de um processo.

```
$ kill -STOP $PID
$ ps -o pid,ppid,state,cmd -p $PID
  PID  PPID S  CMD
18423 17110 T  sleep 300

$ kill -CONT $PID      # volta para S
$ kill $PID            # encerra (TERM)
```

STOP → **T** (suspenso) → CONT → **S** (dormindo) → TERM → encerrado

Experimento 7 de 7: CPU e memória em tempo real

Agora observamos o SO mediando recursos compartilhados por todos.

```
$ top
# ou, se estiver instalado:
$ htop

# observem as colunas:
PID  USER  %CPU  %MEM  S  COMMAND
```

Ações da dupla: abrir navegador e editor → rodar `sleep 300 &` novamente → observar consumo → fechar um programa e ver o efeito.

Pergunta: se todos os processos querem recursos ao mesmo tempo, por que o sistema continua funcionando?

Mini-desafio em duplas

Repetir o ciclo completo sem copiar comandos cegamente, registrando o que foi observado.

Momento	PID	PPID	Estado	Significado
sleep 300 &				
após kill -STOP				
após kill -CONT				
após kill (TERM)				

Entrega oral

Uma dupla explica em 60 segundos o que aconteceu com o processo: não precisa de código, precisa explicar o fenômeno.

Erros comuns no laboratório

“Não achei o PID” / “kill: no such process”

O sleep já terminou (dura só 300s). Crie outro com `sleep 600 &`.

“pstree: command not found”

Use `ps -o pid,ppid,cmd -p $$,$PID`. A relação pai/filho aparece igual.

PID digitado errado

Confirme com `ps` antes de repetir o comando.

“Matei o terminal inteiro”

O PID usado era o do `bash`. Abra um novo terminal e confira sempre a coluna `CMD`.

Regra de ouro

Antes de qualquer `kill`, rode `ps` e confirme que o `CMD` é o processo que você criou.

Transformar a observação do terminal em conceitos de Sistemas Operacionais:

- **Programa $\neq$ processo:** o que mudou no sistema quando digitamos `sleep 300`?
- **Tabela de processos:** onde aparecem PID, PPID, estado e comando?
- **Hierarquia:** como sabemos quem é o pai de um processo?
- **Controle por sinais:** por que `kill -STOP` não encerra o processo?
- **Gerência de recursos:** quem decide CPU, memória e ordem de execução?

Cada dupla responde oralmente a duas dessas perguntas, usando o PID da própria máquina.

Síntese conceitual

O laboratório mostrou quatro ideias centrais da disciplina:

Processo

Programa em execução, com identidade (PID), pai (PPID) e estado.

Sinais

Mecanismo para pedir ao SO que altere o comportamento de um processo.

Tabela de processos

O SO mantém metadados de cada processo para poder controlá-lo.

Escalonamento

A CPU é compartilhada segundo decisões tomadas pelo sistema operacional.

Definição operativa de hoje

O sistema operacional coordena processos e recursos para manter o sistema funcionando de forma controlada.

Scripts: um gostinho

Introdução ao shell scripting

- **Shell script**: pequeno programa escrito em uma linguagem interpretada diretamente pelo shell.
- Automatiza tarefas e simplifica operações repetitivas.
- Shells comuns: **Bash**, Dash, Zsh.

Hoje é só um gostinho

Vamos ver o essencial (variáveis e estruturas de controle) para você reconhecer um script quando vir um.

Variáveis armazenam dados que podem ser usados e manipulados ao longo do script.

```
#!/bin/bash
numero1=5
numero2=10
soma=$((numero1 + numero2))
echo "A soma é: $soma"
```

Estruturas de controle: if, for, while

```
# IF: desvio baseado em uma condicao
if [ $idade -ge 18 ]; then
    echo "Voce e maior de idade."
fi
```

```
# FOR: repete um numero definido de vezes
for i in 1 2 3 4 5; do
    echo "Volta $i"
done
```

```
# WHILE: repete enquanto a condicao for verdadeira
contador=0
while [ $contador -lt 5 ]; do
    echo "Contador: $contador"
    contador=$((contador + 1))
done
```

Comparação numérica: o que cada operador significa

Dentro de `[ ... ]`, comparações numéricas usam abreviações em vez de símbolos como `<` e `>` (que, no Bash, têm outro significado dentro de colchetes).

Operador	Significado	Em inglês
<code>-eq</code>	igual a	equal
<code>-ne</code>	diferente de	not equal
<code>-lt</code>	menor que	less than
<code>-le</code>	menor ou igual a	less or equal
<code>-gt</code>	maior que	greater than
<code>-ge</code>	maior ou igual a	greater or equal

```
if [ $idade -ge 18 ]; then      # idade maior ou igual a 18
if [ $nota -lt 6 ]; then       # nota menor que 6
if [ $a -ne $b ]; then         # a diferente de b
```

Operadores (visão geral)

Aritméticos

+ - * / % **

Comparação (números)

-eq -ne -lt -le -gt -ge

(veja o significado de cada um no slide anterior)

Comparação (strings)

= != < >

Lógicos

&& || !

Usados dentro de [...] ou \$((...)) em condicionais e loops.

Fechamento

1. Crie uma pasta chamada `exercicio` no seu diretório de usuário.
2. Dentro dela, liste os processos em execução do seu usuário e exporte para `proc_usuario.txt`.
3. Liste os processos do usuário `root` e exporte para `proc_root.txt`.
4. Determine a quantidade total de processos em execução no sistema.
5. Inicie um processo com prioridade `-5`, depois atualize para `1` e finalize-o.
6. Execute `ps`, abra o Firefox em segundo plano (`&`), pare-o em outro terminal sem finalizá-lo e depois libere sua execução.

Respostas

```
mkdir exercicio
ps aux | grep $USER > exercicio/proc_usuario.txt
ps aux | grep root > exercicio/proc_root.txt
ps aux | wc -l

nice -n -5 sleep 10 &      # inicia com prioridade -5
renice 1 -p <PID>          # atualiza a prioridade para 1
kill <PID>                 # finaliza o processo

ps                          # lista processos
firefox &                  # abre em segundo plano
kill -STOP <PID>           # pausa (em outro terminal)
kill -CONT <PID>           # libera a execucao
```

Anexo: Cola de comandos

Criar e listar

```
sleep 300 &      # 2o plano  
jobs             # jobs do shell  
fg %1           # traz p/ 1o plano  
ps -f           # tabela atual  
pstree -p $$    # arvore
```

Controlar e monitorar

```
kill -STOP $PID  
kill -CONT $PID  
kill $PID  
top / htop  
echo $$         # PID do shell
```

Nenhum comando desta lista precisa de sudo.

Dúvidas?

Weslei Ferreira Santos
wesleisantos@ufba.br
MATA58 · Sistemas Operacionais
Instituto de Computação / UFBA